

Assignment

A

1) What are arrays in C programming? Wap that demonstrate the use of a single-dimensional array to store and print the marks of 5 students. Explain how arrays are initialized.

→ An array is a collection of elements of the same data type, stored at contiguous memory locations. Array always you to store multiple values using a single variable name.

Example

```
int arr[5]; // Declare an array of 5 integers.
```

program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int marks[5] = {85, 90, 78, 92, 88};
```

```
printf("Marks of 5 students are: \n");
```

```
for (int i=0; i<5; i++)
```

```
{
```

```
printf("Student %d : %.2d \n", i+1, marks[i]);
```

```
}
```

```
return 0;
```

```
}
```

Output: Marks of 5 students are:

Student 1: 85

Student 2: 90

Student 3: 78

Student 4: 92

Student 5: 88

## Syntax for Declaring an Array

`type array_name[size];`

- `type` is the data type of the array (e.g. `int`, `float`)
- `array_name` is the name you give to the array
- `size` is the number of elements the array will hold

Array initialization means assigning values to the elements of an array at the time of declaration.

2) Explain how array elements are stored in memory. Create a program that demonstrates how to access and manipulate array elements, and explain how array bounds checking works in C.

→ Array elements are stored in contiguous memory locations. Each element is placed next to the previous one in memory.

For an array of integers, each element takes 4 bytes and they are stored back-to-back.

Example:

If you declare `int arr[3] = {1, 2, 3}`, the memory layout could look like

- `arr[0]` is at memory address 1000
- `arr[1]` is at memory address 1004

Program:

```
#include <stdio.h>
```

```
int main()
```

```
{ int marks[3] = {85, 90, 78};
```

```
printf("original marks of 3 students: \n");
```

```
for (int i=0; i<3; i++)
```

```
{
```

```
printf("Student %d : %d \n", i+1, marks[i]);
```

```
}
```

```
marks[0] = 95;
```

```
marks[2] = 80;
```

```
printf("\n updated marks after manipulation: \n");
```

```
for (int i=0; i<3; i++)
```

```

}
printf("Student %d: %d\n", i+1, marks[i]);
}

printf("In Attempt to access out of bound elements: \n");
printf("Attempt to access bound marks[3]: %d\n", marks[3]);
return 0;

```

### output

original marks of 3 students

Student 1: 85

Student 2: 90

Student 3: 78

updated marks after manipulation

Student 1: 95

Student 2: 90

Student 3: 80

Attempting to access out of bound elements

Attempt to access marks[3] :- 858993460

### \* Bounds checking in C

- C does not perform automatic bounds checking. If you try to access an index outside the array size, it leads to undefined behaviour.

#### Example

- int arr[3] = {10, 20, 30}; printf("%d", arr[5]);  
error accessing out of bounds, give some garbage value

- 3) Write a program to pass array elements to a function. Explain the difference between passing individual array elements and passing the entire array to a function. Provide examples for both approaches.

→ Program:

```
#include <stdio.h>
```

```
void printElement (int element)
```

```
{
```

```
    printf("Element : %d\n", element);
```

```
}
```

{ passing individual array }

```
int main()
```

```
{ int arr[5] = {10, 20, 30, 40, 50};
```

```
    printElement (arr[2]);
```

```
    return 0;
```

```
}
```

Output

Element = 30

Program: #include <stdio.h>

#include <conio.h>

```
void printElement (int arr[], int size) { passing entire array }
```

```
{
```

```
    for (int i = 0; i < size; i++)
```

```
{
```

```
        printf("%d ", arr[i]);
```

```
}
```

```
printf("\n");
```

```
{
```

```
int main()
```

```
{
```

```
int arr[5] = {1, 2, 3, 4, 5};
```

```
printElement(arr, 5);
```

```
return 0;
```

```
}
```

```
output
```

```
1 2 3 4 5
```

The difference are as follows

|      | Passing individual array                                             | Passing entire array                                               |
|------|----------------------------------------------------------------------|--------------------------------------------------------------------|
| i)   | The value of the specific element is passed i.e. <code>arr[2]</code> | The memory address of the first element of the array is passed     |
| ii)  | only one value is passed at a time                                   | All elements of the array are accessible through indexing          |
| iii) | Changes to the element do not affect the original array.             | Changes to the array inside the function affect the original array |
| iv)  | Example:- <code>printElement(arr[2]);</code>                         | Example:- <code>printElement(arr, 5);</code>                       |

④ What is a two-dimensional array? Write a program to initialize a two-dimensional array and perform matrix addition. Explain how memory mapping of a 2D array is handled in C.

→ A two-dimensional array is like a table (matrix) with rows and columns where each element is identified by two indices.  
program:

```
#include <stdio.h>
int main()
{
    int rows, cols, i, j;
    printf("Enter the number of rows and columns: ");
    scanf("%d %d", &rows, &cols);
    int matrix1[rows][cols], matrix2[rows][cols], sum[rows][cols];
    printf("Enter elements of the first matrix: \n");
    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            printf("matrix1 [%d][%d]: ", i, j);
            scanf("%d", &matrix1[i][j]);
        }
    }
    printf("Enter elements of the second matrix: \n");
    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            printf("matrix2 [%d][%d]: ", i, j);
            scanf("%d", &matrix2[i][j]);
        }
    }
```

```

for (i=0; i < rows; i++)
{
    for (j=0; j < cols; j++)
    {
        sum[i][j] = matrix1[i][j] + matrix2[i][j];
    }
}

printf("Sum of the matrices: \n");
for (i=0; i < rows; i++)
{
    for (j=0; j < cols; j++)
    {
        printf("%d", sum[i][j]);
    }
    printf("\n");
}

return 0;

```

Output:

Enter the number of rows and columns: 2

2

Enter elements of the first matrix:

matrix1[0][0]: 1

matrix1[0][1]: 2

matrix1[1][0]: 3

matrix1[1][1]: 4

Enter elements of the second matrix:

matrix2[0][0]: 5

matrix2[0][1]: 6

matrix 2[0][0] = 7

matrix 2[1][0] = 8

Sum of the matrices

6 8

10 12

Memory mapping in 2D array

- In memory, 2D arrays are stored row by row  
example

int arr[2][3] = { {1, 2, 3}, {4, 5, 6} };

memory layout

Address | value

1000 | 1 (arr[0][0])

1004 | 2 (arr[0][1])

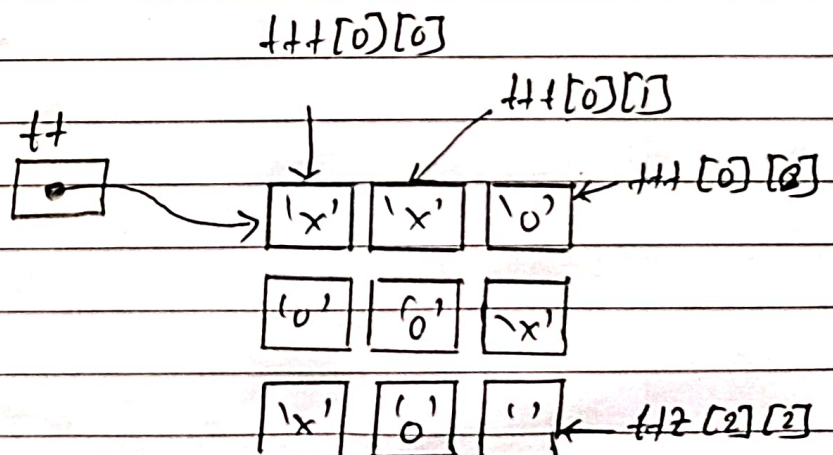
1008 | 3 (arr[0][2])

1012 | 4 (arr[1][0])

1016 | 5 (arr[1][1])

1020 | 6 (arr[1][2])

Diagram:



- ⑤ Discuss the use of pointers with two-dimensional arrays. Write a program that use of pointers with ~~two-dimensional~~ to access and manipulate a two-dimensional array. Explain how pointers can be used to pass 2D array to a function.

→ The name of a 2D array is a pointer to its first row. `arr` points to `arr[0]`. and `arr[0]` points to the first element.

Program:

```
#include <stdio.h>
int main ()
{
    int arr [2][3] = {{1,2,3}, {4,5,6}};
    printf("Address of arr = %p\n", arr);
    printf("Address of arr[0] = %p\n", arr[0]);
    printf("Value at arr[0][0] = %d\n", *((arr + 0) + 1));
    // accessing arr[0][0]
    return 0;
}
```

Output

Address of arr = 0x7ffe0a

Address of arr[0] = 0x7ffeb

Value at arr[0][0] = 2

★ Pointers to pass 2D array to functions

- We can pass a 2D array to a function by specifying the number of columns in the function parameter.

Program

```
#include <stdio.h>
```

```
void print Array (int arr [2][3])
```

```
{
```

```
    for (int i=0; i<2; i++)
```

```
    {
```

```
        for (int j=0; j<3; j++)
```

```
        {
```

```
            printf("%d", arr[i][j]);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
int main()
```

```
{
```

```
    int arr [2][3] = {{1,2,3}, {4,5,6}};
```

```
    print Array (arr);
```

```
    return 0;
```

```
}
```

Output

1 2 3

4 5 6

Here we see how array of 2D is passed to a function